

Evan Baesler
October 4, 2026
31151619

EE Design 1 Technical Report

Microcontroller ADC & LCD

Introduction

In this module, our goal was to create an ohmmeter through a Raspberry Pi Pico 2 and an LCD. To do this we had to use our ADCs and a voltage divider with one known resistor and one unknown resistor that we would use to test. The margins for this design were to be within 5% of the resistance of any resistance between 1kOhm and 1MOhm.

Methods

The first step in designing our ohmmeter was to define the hardware scope. For this project, we knew we would need an LCD, Raspberry Pi Pico 2, potentiometers for brightness and contrast, and a voltage divider to determine the resistance of the unknown resistor.

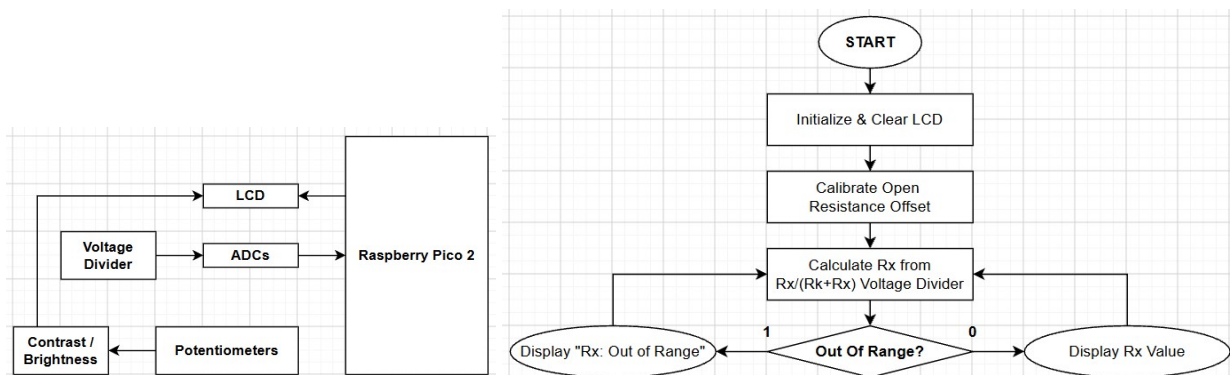


Figure 1: Hardware (Left) and Software (Right) Flowchart / Functional Block Diagram.

The resistance of our unknown resistor is equivalent to:

$$R_x = R_k * \frac{V_x}{V_{in} - V_x}$$

Which gives us the insight that we can apply that formula to our ADC to derive the value of R_x . While the fundamental design is elementary on a surface level, there is a lot of nuance to the implementation. For example, some calibration issues we run into are:

- Power supply and regulation limitations of the Raspberry Pi Pico 2
- Electrical Noise
- Accuracy of known resistors
- ADC quantization error and linearity

From most severe to least, we first notice a large drop in the power supply of the Raspberry Pi Pico 2, which can quickly cause our rails to fluctuate, which can massively affect the program when hard-coded for an expected rail of 3.3V; so we use an exterior power supply with higher current capacity.

Then, we see electrical noise on the ADCs & rail, so we apply 100nF capacitors to bypass any electrical noise straight to ground. This only requires two capacitors as ADC_Vin is shorted to VCC, meaning that the capacitor there shorts both VCC and ADC_Vin to GND; while ADC_Vx requires a separate bypass.

Along with this, we find that the R_k must be inspected for its true value, as there are tolerances from the manufacturer that must be accounted for. In this model, the exact resistance found was 37580 Ohms on a 37K.

For the ADC, the first thing we find is ADC input noise is a major factor of our design (1), as there is kT/C resistor noise, and there are limitations on the amount of bits that can be free of noise. This works with and against us in some instances, as some noise is good as it allows the results to ever-so-slightly fluctuate as to allow for averages between two values to happen, otherwise; the results, if slightly offset, would remain fixed with that offset. We also find that a 12-bit ADC can represent 4096 values, which means that our minimum voltage step is $3.3/4096 = 0.00080566406V$, which led to limitations on the higher end of our spectrum. This, combined with a high input impedance on the ADC when we reach higher loads meant that higher R_x values led to unreliable outputs versus expectations.

To compensate, resistors of known sizes were placed on the board after calibration and a piecewise function was used to tune the values read to the actual values, which significantly



Results

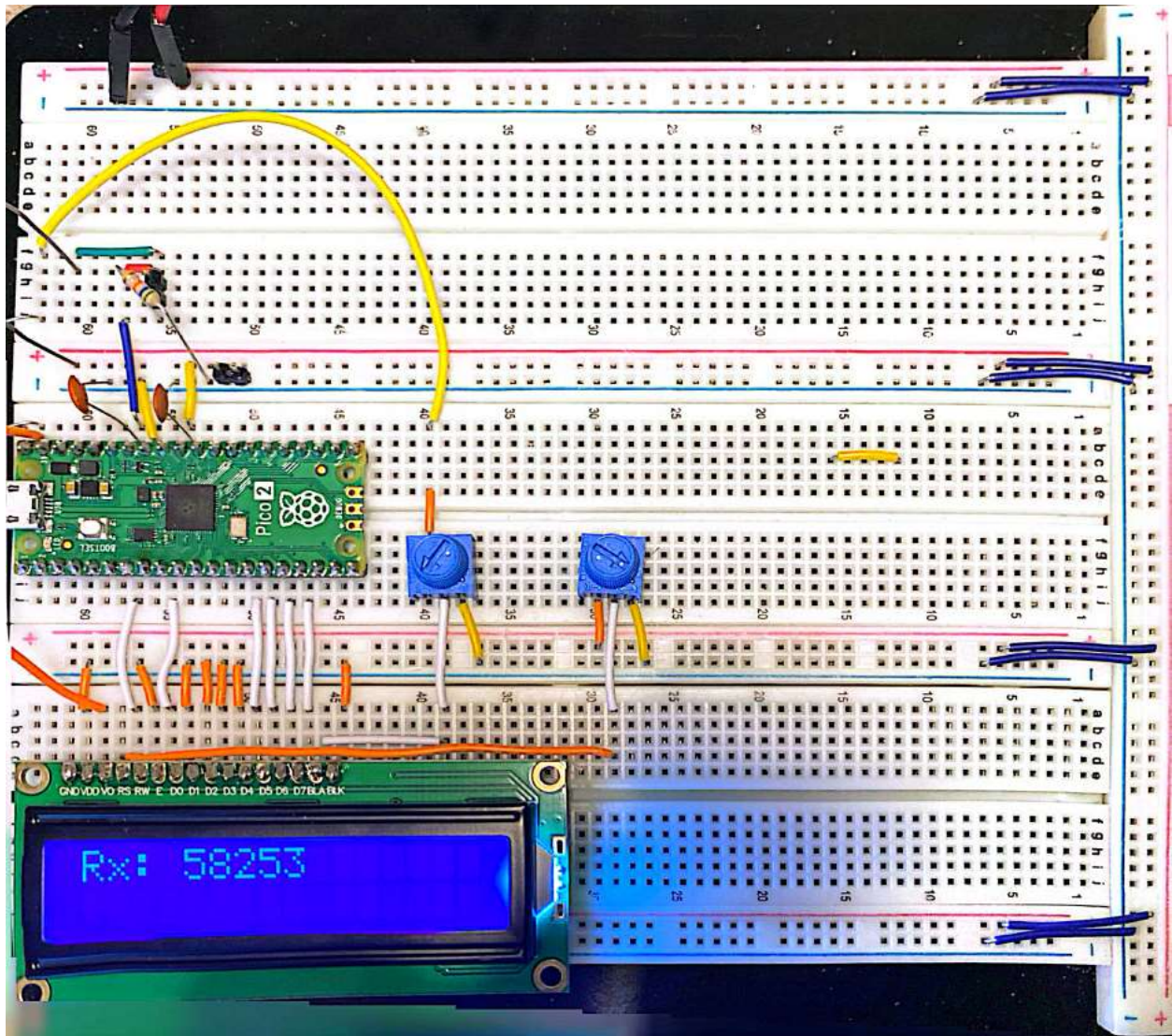


Figure 3: Picture of Ohmmeter in Action

As mentioned above, to calibrate our ohmmeter, we utilized an open circuit offset, a piecewise function, bypass capacitors, external voltage regulation and power supply.

Conclusion

This module reinforced our design capabilities by making us think from the ground up about making something as commonly used as an ohmmeter and showed the full hardware to software to implementation pipeline and how to refine on the go. This module also helped us realize the actual implementation of commercial grade oscilloscopes, with multiple comparison resistors for different ranges, noise compensation, powerful supplies, and internal transistor circuitry to automate the switching processes.

Appendix

CFAH1602Z-YYH-ET LCD Firmware

Written by Evan Baesler

September 26, 2026

For: Raspberry Pi Pico 2

from machine import ADC, Pin

import utime

Pins are in terms of GPIO Ports

RW, D0-D3 to GND

LEGEND:

XP = External Potentiometer Control

Contrast Control (XP)

Backlight Brightness (XP)

Chip Enable

EN = Pin(6, Pin.OUT) # Enable to push data into LCD

Register Select

RS = Pin(5, Pin.OUT) # 0 = command | 1 = data

Data bus (lower bits are not utilized in 4-bit mode!)

D4 = Pin(10, Pin.OUT)

D5 = Pin(11, Pin.OUT)

D6 = Pin(12, Pin.OUT)

D7 = Pin(13, Pin.OUT)

ADC Pins

adc_vx = ADC(28)

adc_vin = ADC(26)

PORT = [10, 11, 12, 13]

L = [0,0,0,0]

```
## SUBROUTINES
```

```
def configure():
```

```
    for i in range(4):
```

```
        L[i] = Pin(PORT[i], Pin.OUT)
```

```
def lcd_strobe():
```

```
    EN.value(1) # Send Data to LCD
```

```
    utime.sleep_ms(1) # Sleep for 1 ms
```

```
    EN.value(0) # Read Command / Data
```

```
    utime.sleep_ms(1) # Sleep for 1 ms
```

```
# ASCII is an 8-bit data format, so we need to do
```

```
# the upper and lower nibble individually
```

```
def lcd_write(c, mode):
```

```
# FIRST NIBBLE (upper 4 bits)
```

```
    if mode == 0: # Command in numerical form
```

```
        d = c
```

```
    else:
```

```
        d = ord(c) # Convert Hexadecimal to ASCII
```

```
    d = d >> 4 # Shift right to access upper nibble
```

```
    for i in range(4): # Put bits to D4-D7
```

```
        b = d & 1 # Extract lowest bit
```

```
        L[i].value(b) # Set that GPIO pin
```

```
        d = d >> 1 # Shift right to next bit
```

```
    RS.value(mode) # Set RS via mode
```

```
    lcd_strobe() # Pulse EN to latch upper byte
```

```
# SECOND NIBBLE (lower 4 bits)
```

```
    if mode == 0: # Command in Hexadecimal
```

```
        d = c
```

```
    else:
```

```
        d = ord(c) # Convert Hexadecimal to ASCII
```

```
    for i in range(4): # Put bits to D4-D7
```

```
        b = d & 1 # Extract lowest bit
```

```
        L[i].value(b) # Set that GPIO pin
```

```
d = d >> 1 # Shift right to next bit
```

```
RS.value(mode) # Set RS via mode
```

```
lcd_strobe() # Pulse EN to latch upper byte
```

```
utime.sleep_ms(1)
```

```
RS.value(1) # Return RS to data mode
```

```
def lcd_clear():
```

```
    lcd_write(0x01, 0)
```

```
    utime.sleep_ms(5)
```

```
def lcd_home():
```

```
    lcd_write(0x02, 0)
```

```
    utime.sleep_ms(5)
```

```
def lcd_cursor_blink():
```

```
    lcd_write(0x0D, 0)
```

```
    utime.sleep_ms(1)
```

```
def lcd_cursor_on():
```

```
    lcd_write(0x0E, 0)
```

```
    utime.sleep_ms(1)
```

```
def lcd_cursor_off():
```

```
    lcd_write(0x0C, 0)
```

```
    utime.sleep_ms(1)
```

```
def lcd_puts(s):
```

```
    l = len(s)
```

```
    for i in range(l):
```

```
        lcd_putchar(s[i])
```

```
def lcd_putchar(c):
```

```
    lcd_write(c, 1)
```

```
def lcd_goto(col, row):
```

```
    c = col + 1
```

```
    if row == 0:
```

```
        address = 0
```

```
    if row == 1:
```

```
    address = 0x40
if row == 2:
    address = 0x14
if row == 3:
    address = 0x54
address = address + c - 1
lcd_write(0x80 | address, 0)
```

```
def lcd_init():
    configure()
    RS.value(0)
    EN.value(0)
    utime.sleep_ms(120)
    for i in range(4):
        L[i].value(0)
    utime.sleep_ms(50)
    L[0].value(1)
    L[1].value(1)
    lcd_strobe()
    utime.sleep_ms(10)
    lcd_strobe()
    utime.sleep_ms(10)
    lcd_strobe()
    utime.sleep_ms(10)
    L[0].value(0)
    lcd_strobe()
    utime.sleep_ms(5)
    lcd_write(0x28, 0)
    utime.sleep_ms(1)
    lcd_write(0x08, 0)
    utime.sleep_ms(1)
    lcd_write(0x01, 0)
    utime.sleep_ms(10)
    lcd_write(0x06, 0)
    utime.sleep_ms(5)
    lcd_write(0x0C, 0)
    utime.sleep_ms(10)
```

```
# ADC (returns averaged raw counts, 0-65535)
```

```
def adc_read_both(samples=100000):
```

```
    vin_total = 0
```

```
    vx_total = 0
```

```
    for i in range(samples):
```

```
        vin_total += adc_vin.read_u16()
```

```
        vx_total += adc_vx.read_u16()
```

```
    return vin_total / samples, vx_total / samples
```

```
def calculate_resistance(vin, vx, offset=0):
```

```
    rk = 37580
```

```
    if (vin - vx) > 0:
```

```
        rx_unscaled = ((rk * vx) / (vin - vx)) - offset
```

```
    if rx_unscaled > 6000000: # 1M
```

```
        rx = rx_unscaled * 1000/7000
```

```
    elif rx_unscaled > 3500000: # 900K
```

```
        rx = rx_unscaled * 900/4050
```

```
    elif rx_unscaled > 2500000: # 800K
```

```
        rx = rx_unscaled * 800/2700
```

```
    elif rx_unscaled > 1500000: # 700K
```

```
        rx = rx_unscaled * 700/1850
```

```
    elif rx_unscaled > 1000000: # 600K
```

```
        rx = rx_unscaled * 600/1355
```

```
    elif rx_unscaled > 700000: # 500K
```

```
        rx = rx_unscaled * 500/960
```

```
    elif rx_unscaled > 500000: # 400K
```

```
        rx = rx_unscaled * 400/671
```

```
    elif rx_unscaled > 290000: # 300K
```

```
        rx = rx_unscaled * 300/445
```

```
    elif rx_unscaled > 190000: # 200K
```

```
        rx = rx_unscaled * 200/265
```

```
    elif rx_unscaled > 90000: # 100K
```

```
        rx = rx_unscaled * 100/120
```

```
    elif rx_unscaled > 35000: # 40K
```

```
        rx = rx_unscaled * 40000/45500
```

```

    elif rx_unscaled > 20000: # 20K
        rx = rx_unscaled * 20000/22359
    elif rx_unscaled > 9000: # 10K
        rx = rx_unscaled * 10000/11000
    elif rx_unscaled > 900: # 1K
        rx = rx_unscaled * 1000/1110
    elif rx_unscaled < 400: # Too low
        return None, None
    else:
        rx = rx_unscaled

    return rx, rx_unscaled

return None, None

def adc_calibrate():
    lcd_clear()
    lcd_puts("Short Terminals!")
    utime.sleep_ms(1000)

    lcd_clear()
    lcd_puts("Calibrating...")

    vin, vx = adc_read_both(10000)
    rx, rx_unscaled = calculate_resistance(vin, vx, 0)

    lcd_clear()

    return rx_unscaled if rx_unscaled is not None else 0

def lcd_display_resistance(offset):
    vin, vx = adc_read_both(10000)
    rx, rx_unscaled = calculate_resistance(vin, vx, offset)

    lcd_clear()

    if rx is None or rx > 1000000:
        lcd_puts("Rx: Out of Range")
    # lcd_goto(0, 1)

```

```

# lcd_puts("Raw: Out of Range")
else:
    rx = max(0, rx)
    rx_unscaled = max(0, rx_unscaled)

    lcd_puts("Rx: " + str(int(rx)))
# lcd_goto(0, 1)
# lcd_puts("Raw: " + str(int(rx_unscaled)))
## END OF SUBROUTINES

# PROGRAM START

lcd_init()
lcd_clear()

offset = adc_calibrate()

while True:
    lcd_display_resistance(offset)
    utime.sleep_ms(500)

# PROGRAM END

```

References

1. <https://www.analog.com/en/resources/analog-dialogue/articles/adc-input-noise.html>
2. ChatGPT / Google Gemini